

Course Project Guidelines

DSCI 551 | Fall 2026

*Proposal Due: **Friday, October 9, 2026 at 11:59 PM** | Demo: Tuesday, November 24 and Tuesday, December 1, 2026 (in class) | Final Report Due: **Thursday, December 10, 2026 at 11:59 PM***

Foundations of Data Management

Real-World Data Management, Database Design, and Query Processing

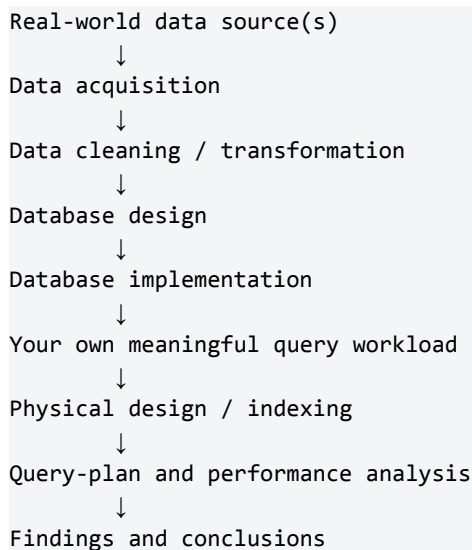
1. Project Overview

The course project gives you an opportunity to apply the database concepts covered in this course to a real-world data management problem.

You may work individually or in a team of up to three students.

You will identify a problem of interest, obtain a meaningful real-world dataset—preferably through one or more public APIs or open-data sources—and design and implement a database system for managing and querying the data.

The emphasis of the project is on database design, implementation, physical database design, and query processing, rather than simply performing data analysis or building a polished user interface.



2. Project Topic and Data

You may choose an appropriate application domain subject to instructor approval.

- weather and climate;
- earthquakes and geological events;
- GitHub repositories and software-development activity;
- government spending;

- transportation and flight data;
- housing and real estate;
- university and education data;
- public health;
- sports;
- movies and entertainment;
- employment and labor-market data;
- other appropriate public/open datasets.

Possible data sources include the USGS Earthquake API, Open-Meteo, GitHub API, USAspending, and other appropriate public APIs or open-data repositories.

You may combine multiple data sources when appropriate. Your project should contain enough data and sufficiently interesting relationships and queries to support meaningful database design and query-processing analysis.

You are not required to collect millions of records merely to make the project large. The data should be large and rich enough for the database concepts you investigate to be meaningful.

3. MySQL Component

MySQL will normally be the primary database system for your project.

- Design an appropriate relational schema.
- Identify entities, attributes, relationships, primary keys, and foreign keys.
- Choose appropriate MySQL data types and constraints.
- Implement your database in MySQL.
- Develop a reproducible process for loading your data.
- Design meaningful queries motivated by your application.
- Identify and evaluate appropriate indexes.
- Examine query execution using EXPLAIN and/or EXPLAIN ANALYZE.
- Connect observed query behavior to physical database and query-processing concepts discussed in class.

Formal normalization analysis is not a major project requirement. However, your schema should be reasonable for the application, and you should be able to explain important design decisions.

4. Your Query Workload

You should design your own query workload. Do not create artificial queries solely to satisfy a checklist. Your queries should answer meaningful questions about your application or data.

Collectively, your workload should involve several important database operations, such as:

- selection and filtering;
- multi-table joins;
- aggregation;
- GROUP BY;
- ORDER BY;
- queries that may benefit from indexes.

For each query selected for detailed analysis, explain what real-world question the query answers before discussing its SQL or execution plan.

5. Physical Database Design and Query Analysis

A major objective of the project is to connect your queries to the physical database concepts studied in class.

You should select representative queries from your own workload for detailed analysis. Depending on your queries, your analysis may involve:

- table scan vs. index access;
- B+ tree indexes;
- index selectivity;
- single-column vs. composite indexes;
- column order in a composite index;
- sorting;
- index-provided ordering;
- internal/external sorting;
- joins;
- possible join algorithms;
- relation sizes;
- predicate selectivity;
- query execution plans;
- measured execution performance.

```
Original query / physical design
    ↓
EXPLAIN / EXPLAIN ANALYZE
    ↓
Interpret the execution plan
    ↓
Propose an index or physical-design change
    ↓
Run the query again
    ↓
Compare plan and performance
    ↓
Explain why the result changed
```

You are not expected to force MySQL to use every algorithm discussed in class. Instead, where appropriate, discuss which algorithms could theoretically implement an operation and compare that reasoning with the plan MySQL actually chooses.

6. What Does “Deeper MySQL Physical-Design Analysis” Mean?

If your team chooses the deeper-analysis option, you should investigate alternative ways to physically support your workload and explain how those choices affect query execution. This does not simply mean writing more SQL queries.

For example, for a query involving filtering and ordering, you might compare several index designs:

```
No additional index
    vs.
INDEX(magnitude)
    vs.
INDEX(event_time)
    vs.
INDEX(magnitude, event_time)
```

- Which index helps the selection condition?
- Which index may help the requested ordering?
- Does MySQL perform an explicit sort?
- Can an index provide the requested order?
- Why does MySQL choose one access method instead of another?
- How does predicate selectivity affect the plan?
- Does adding an index actually improve execution time?
- Why might MySQL choose not to use an available index?

You may also vary workload conditions, such as query selectivity, and investigate whether the physical plan changes.

For a join query, you may compare different indexing conditions, relation sizes, or selectivities and relate the resulting plan to join algorithms discussed in class.

For a query involving ORDER BY, you may investigate explicit sorting versus index-provided ordering and relate an explicit sort to the internal/external sorting concepts discussed in class.

Deeper analysis means comparing alternative physical designs and workload conditions, examining their execution plans and measured performance, and explaining the results using concepts from class.

7. NoSQL Component

Because NoSQL is not the primary emphasis of the course, the NoSQL component is intended to be relatively modest.

If your project includes a NoSQL component, you may use MongoDB or Firebase.

You do not need to reproduce your entire MySQL system in NoSQL. Instead, choose a meaningful subset of your data or application and demonstrate:

- how the data is represented;
- how the representation differs from your relational design;
- how representative operations are performed;
- advantages and disadvantages of the two approaches for your application.

The goal is to demonstrate that you understand important differences between relational and NoSQL data management, not to build a large production-quality NoSQL system.

8. User Interface / Application

Your project should include a simple user interface or application layer that allows you to demonstrate important database operations and queries.

The interface does not need to be sophisticated. Appropriate options include:

- a simple web application;
- a Streamlit application;
- a Jupyter notebook with interactive inputs;
- a command-line or menu-driven application;
- another simple interface approved by the instructor.

Your interface should support several meaningful operations for your application, such as:

- entering search or filtering criteria;
- executing representative database queries;
- displaying query results clearly;
- showing summaries or simple visualizations where appropriate;
- performing inserts or updates if these operations are meaningful for your application.

The interface must retrieve its results from your database rather than using hard-coded results.

Frontend design is not a major grading criterion. A simple, functional interface is sufficient. You should spend most of your project effort on data management, database design, querying, physical design, indexing, and query-processing/performance analysis.

A full CRUD application is not required unless CRUD operations are naturally important to your application.

9. Expectations by Team Size

Your expected project scope depends on your team size.

Requirement	1 Student	2 Students	3 Students
Real-world data acquisition	Required	Required	Required
MySQL implementation	Required	Required	Required
Student-designed query workload	Required	Required	Required
Index/query-plan analysis	Required	Required	Required
NoSQL	Optional	NoSQL or deeper MySQL analysis	Required, modest scope
Physical-design analysis	Appropriate indexes + before/after comparison	Compare multiple alternative physical designs	Multiple queries + multiple physical-design alternatives
Query-processing analysis	Explain representative execution plans	Compare plans under alternative designs/conditions	Detailed analysis across multiple query-processing operations

How to interpret the analysis expectations

Physical-design analysis asks what indexes or other physical design choices should support your workload. For a solo project, choose appropriate indexes and compare query plans/performance before and after. A two-person team should compare multiple alternatives (for example, single-column vs. composite indexes, column order, or different selectivities). A three-person team should examine multiple representative queries and several physical-design alternatives, including interactions among filtering, ordering, and joins.

Query-processing analysis asks how MySQL actually executes your queries and why. A solo project should explain representative EXPLAIN / EXPLAIN ANALYZE plans and major operations such as scans, index access, joins, and sorting. A two-person team should compare plans under alternative physical

designs or workload conditions. A three-person team should perform detailed analysis across several query-processing operations and connect MySQL's choices to indexing, sorting, join algorithms, selectivity, and I/O/cost concepts discussed in class.

10. Phase I — Project Proposal — Due Friday, October 9, 2026 at 11:59 PM

The proposal describes what your team plans to build and why it is appropriate for this course.

The proposal should include:

A. Team

- team member names;
- proposed division of responsibilities.

B. Problem and Motivation

- your application/domain;
- the problem you want to address;
- why the data is interesting;
- intended users or use cases, if applicable.

C. Data Sources

- source/API name;
- URL or documentation;
- type and format of the data;
- important fields;
- approximate expected data volume;
- API restrictions or rate limits, if applicable;
- how you plan to collect the data.

D. Preliminary Database Design

- major entities;
- important relationships;
- a preliminary ER diagram;
- an initial relational schema.

E. Proposed Query Workload

Identify several questions your database should be able to answer. At the proposal stage, focus on the questions rather than finalized SQL.

F. Potential Physical-Design Opportunities

- indexing;
- sorting;
- joins;
- query performance.

G. Project Scope

Explain how your planned project scope is appropriate for your team size. If you plan to include MongoDB or Firebase, briefly describe the proposed NoSQL component.

11. Phase II — Project Demo — Tuesday, November 24 and Tuesday, December 1, 2026 (in class)

Your demo should demonstrate a working database system, not simply a slide presentation.

A. Data Acquisition

Demonstrate the overall acquisition/transformation/loading pipeline. You do not need to explain every line of ingestion code.

B. Database

- implemented schema;
- representative data;
- important relationships;
- approximate database size.

C. Query Workload

Run several meaningful queries and explain the application questions they answer.

D. Physical-Design Experiment

Demonstrate at least one meaningful physical-design experiment, such as comparing a query plan before and after adding or changing an index.

E. Query Processing

Discuss at least one query involving an important operation such as index-based access, sorting, joining, or aggregation. Connect your observations to concepts discussed in class.

F. NoSQL Component, if applicable

Briefly demonstrate your MongoDB/Firebase component and explain how its representation or operations differ from MySQL.

G. Team Understanding

All team members should be prepared to answer questions about the entire project, not only the portion they personally implemented.

H. Demo readiness:

All required project implementations must be **completed and working before the assigned demo**.

The demo is not for presenting unfinished core functionality.

12. Phase III — Final Report — Due Thursday, December 10, 2026 at 11:59 PM

Your final report should explain both what you built and what you learned from your database experiments.

A. Introduction

- problem;
- motivation;
- application goals.

B. Data

- data sources;
- acquisition process;
- data characteristics;
- cleaning/transformation;
- final database size.

C. Database Design

- ER model;
- relational schema;
- keys and relationships;
- important design decisions.

D. Query Workload

For important queries, state the application question, show the SQL, and explain the important database operations involved.

E. Physical Database Design

- indexes considered;
- indexes implemented;
- why you selected them;
- alternative physical designs considered.

F. Query-Processing and Performance Analysis

This should be one of the most important sections of your report. For representative queries, show relevant EXPLAIN / EXPLAIN ANALYZE results, explain important parts of the execution plan, compare alternative physical designs where appropriate, provide measured performance results, and explain why performance changed.

Where appropriate, discuss B+ tree indexing, selectivity, sorting, index-provided ordering, external sorting, joins, alternative join algorithms, and I/O or cost considerations.

G. NoSQL Component, if applicable

- MongoDB/Firebase representation;
- representative operations;
- differences from MySQL;
- advantages and disadvantages for your workload.

H. Findings and Lessons Learned

- important database-design findings;
- interesting performance results;
- unexpected optimizer behavior;
- limitations;

- possible improvements.

I. Team Contributions

For team projects, clearly describe each member's major contributions. Everyone should understand the complete project even if individual members take primary responsibility for particular components.

J. Code documentation and Google Drive

Students must organize/document their project code, upload the relevant code and reproducibility materials to a Google Drive folder, and include a **shareable Google Drive link in the final report**.

13. Evaluation

The project will be evaluated primarily on the quality of your database design, implementation, physical-design reasoning, and query-processing analysis.

Component	Weight
Proposal	5%
Database/data design and implementation	20%
Query workload	10%
Physical design and indexing	15%
Query-processing/performance analysis	20%
NoSQL / additional depth, as appropriate	5%
User interface	5%
Demo	10%
Final report	10%

The same broad grading categories apply to different team sizes, but the expected scope and depth increase for larger teams. The quality and depth of your analysis matter more than simply producing many queries.

A sophisticated web interface or visualization is welcome, but application polish is not a substitute for substantive database work.